

北京邮电大学信息与知识获取课程实验报告

——信息检索系统

小组成员：赵宇鹏

学号：2022211119

班级：2022211304

北京邮电大学信息与知识获取课程实验报告——信息检索系统

1. 作业要求

1.1 基本要求

1.2 扩展要求

2. 项目介绍

2.1 项目结构

2.2 数据获取

2.3 文件概述

2.3.1 process.py

2.3.1.1 词向量模型

2.3.1.2 倒排索引

2.3.2 search.py

2.3.2.1 搜索匹配算法

2.3.2.2 BM25相关评分

2.3.2.3 短语匹配增强

2.3.2.4 评估与排序算法

2.3.3 main.py

2.4 创新以及算法优化

2.4.1 多媒体实现

2.4.2 缓存加速

2.4.3 搜索算法优化

2.4.4 评估算法优化

2.4.5 评估算法反馈

3. 结果展示

4. 实验总结

1. 作业要求

1.1 基本要求

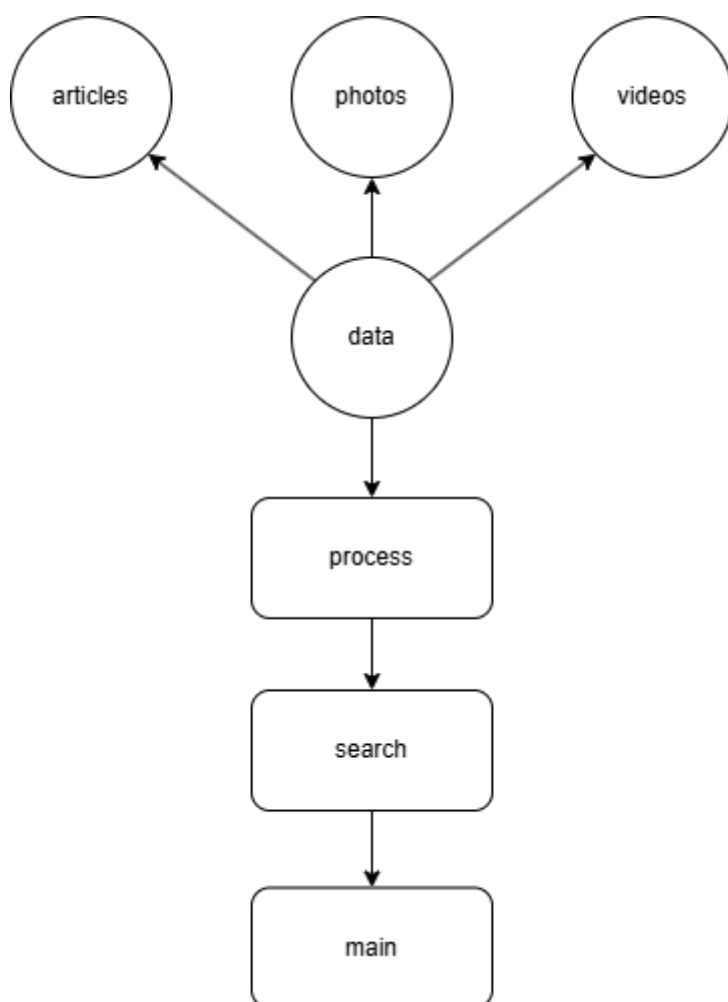
设计并实现一个信息检索系统，中、英文皆可，数据源可以自选，数据通过开源的网络爬虫获取，规模不低于100篇文档，进行本地存储。中文可以分词（可用开源代码），也可以不分词，直接使用字作为基本单元。英文可以直接通过空格分隔。构建基本的倒排索引文件。实现基本的向量空间检索模型的匹配算法。用户查询输入为自然语言字符串，查询结果输出按相关度从大到小排序，列出相关度、文档题目、主要匹配内容、URL、文档日期等信息。最好能对检索结果的准确率进行人工评价。界面不做强制要求，可以是命令行，也可以是可操作的界面。

1.2 扩展要求

鼓励有兴趣和有能力的同学积极尝试多媒体信息检索以及优化各模块算法，也可关注各类相关竞赛。自主开展相关文献调研与分析，完成算法评估、优化、论证创新点的过程。

2. 项目介绍

2.1 项目结构



2.2 数据获取

本项目的数据集，全部都由BBC网站上爬取下来，分为 `articles`、`photos`、`videos` 三部分，其中 `articles` 包含98篇BBC新闻，`videos` 包含64篇新闻，由于在爬取时视频无法储存，所以将含视频的新闻放到此类，`photos` 则包含5张图片，对应 `articles` 中的5篇新闻，以作多媒体示例。

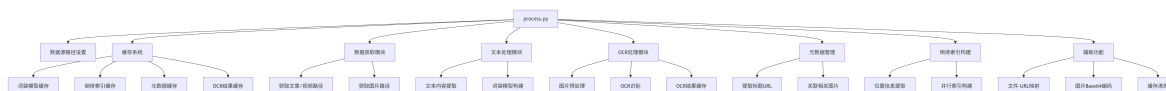
2.3 文件概述

本项目由三个 `py` 文件组成，分别是：

- `process.py`: 负责数据处理以及倒排索引的构建。
- `search.py`: 负责进行信息检索操作。
- `main.py`: 整个项目的入口，负责实现可视化以及调用其他模块的函数来执行整个项目。

2.3.1 process.py

process.py 的主要结构如下图所示:



读取文件以及缓存管理模块的实现相对比较简单，在这里不做过多说明，主要介绍词袋模型的构建以及倒排索引的生成：

2.3.1.1 词向量模型

实现代码如下：

```

1 def get_bag(texts):
2     """构建词袋模型 - 使用缓存"""
3     if os.path.exists(BAG_CACHE) and os.path.exists(COUNT_CACHE):
4         try:
5             with open(BAG_CACHE, 'rb') as f:
6                 bag = pickle.load(f)
7             with open(COUNT_CACHE, 'rb') as f:
8                 count = pickle.load(f)
9             print("已从缓存加载词袋模型")
10            return bag, count
11        except Exception as e:
12            print(f"加载词袋模型缓存时出错: {e}")
13
14    print("构建新的词袋模型...")
15    bag = CountVectorizer(
16        token_pattern=r'\b\w+\b',          # 匹配单词边界的词
17        stop_words='english',              # 去除英文停用词
18        min_df=2,                          # 至少在2个文档中出现
19        max_df=0.95                        # 在超过95%文档中出现的词被视为停用词
20    )
21    count = bag.fit_transform(texts)
22
23    # 保存到缓存
24    try:
25        with open(BAG_CACHE, 'wb') as f:
26            pickle.dump(bag, f)
27        with open(COUNT_CACHE, 'wb') as f:
28            pickle.dump(count, f)
29    except Exception as e:
30        print(f"保存词袋模型缓存时出错: {e}")
31
32    return bag, count

```

为了加速处理，因此特地设置了缓存，若已经存在缓存则直接加载相应的词向量模型，免去每次重启后重新生成模型带来的延迟，本项目基于 [词袋模型](#)，关键实现代码为：

```

1 bag = CountVectorizer(
2     token_pattern=r'\b\w+\b',          # 匹配单词边界的词
3     stop_words='english',              # 去除英文停用词
4     min_df=2,                          # 至少在2个文档中出现
5     max_df=0.95                        # 在超过95%文档中出现的词被视为停用词
6 )

```

主要调用 `scikit-learn` 库中的 `CountVectorizer` 进行实现，其中 `token_pattern` 设置为采用空格分隔，然后设置参数 `stop_words` 为 `english` 自动过滤英语中常见的英语停用词，包括 `the`、`i` 等，然后设置最小文档频率为2，过滤某些可能存在的拼写错误而导致部分仅在单个文档中出现的词语以及某些极少见词汇，设置最大文档频率为0.95，排除那些超高频词汇，防止常见的虚词影响文档区分度以及减慢计算效率。

最后使用 `count = bag.fit_transform(texts)` 方法实现了文本数据到数值向量的关键转换，通过扫描所有文档学习词汇表并构造文档-词项矩阵，该操作首先使用预设的正则表达式 `r'\b\w+\b'` 对文本进行分词处理，匹配单词边界来提取英文词汇并保留数字字符，随后应用内置英文停用词表过滤“the”、“is”等高频虚词，同时依据设定的最小文档频率 `min_df=2` 排除仅出现单次的噪声词，再通过最大文档频率 `max_df=0.95` 阈值滤除超过95%文档共有的泛化词汇，最终生成一个 `scipy.sparse.csr_matrix` 类型的稀疏矩阵，其中每行对应一个文档，每列对应词汇表中的一个词项，矩阵元素值记录各词在文档中的出现频率，这种向量空间表示将原始文本转化为可用于机器处理的数值形式，为后续的倒排索引构建和相关性排序计算奠定了结构化的数据基础。

2.3.1.2 倒排索引

```

1 def generate_inverse_index(text_list, bag, count):
2     """生成倒排索引 - 使用缓存和并行处理"""
3     # 检查缓存
4     if os.path.exists(INDEX_CACHE):
5         try:
6             with open(INDEX_CACHE, 'rb') as f:
7                 return pickle.load(f)
8         except Exception as e:
9             print(f"加载倒排索引缓存时出错: {e}")
10
11     print("生成新的倒排索引...")
12     start_time = time.time()
13     result = defaultdict(dict)
14
15     features = bag.get_feature_names_out()
16
17     tasks = []
18     for i, word in enumerate(features):
19         for j in range(count.shape[0]):
20             if count[j, i] > 0:
21                 tasks.append((word, j, text_list[j]))
22
23     print(f"开始并行处理 {len(tasks)} 个索引任务...")
24     processed = 0
25
26     num_workers = min(os.cpu_count() or 4, 8) # 最多8个进程
27
28     with multiprocessing.Pool(processes=num_workers) as pool:
29         for res in pool.imap_unordered(process_word_positions, tasks,
chunksize=100):

```

```

30         processed += 1
31         if processed % 1000 == 0:
32             print(f"已处理 {processed}/{len(tasks)} 任务
({processed/len(tasks)*100:.1f}%")
33
34         if res:
35             word, doc_idx, value = res
36             result[word][doc_idx] = value
37
38     print(f"倒排索引生成完成, 用时 {time.time() - start_time:.2f} 秒")
39
40     try:
41         with open(INDEX_CACHE, 'wb') as f:
42             pickle.dump(result, f)
43     except Exception as e:
44         print(f"保存倒排索引缓存时出错: {e}")
45
46     return result

```

首先定义了一个名为 `result` 的 `defaultdict` 对象，用于存储生成的倒排索引。`defaultdict` 是Python中的一种特殊字典，它会自动为不存在的键创建一个默认值，这里使用 `dict` 作为默认值类型（因为嵌套两层：词->文档->信息）。

然后，函数通过 `bag.get_feature_names_out()` 获取词袋模型中所有特征词（单词）列表 `features`。接下来，准备一个任务列表 `tasks`，对于每个单词 `word` 和每个文档 `j`（如果该文档中包含该单词，即 `count[j, i] > 0`），我们将 `(word, j, text_list[j])` 加入任务列表。这里 `text_list[j]` 是文档的原始文本。

随后，我们使用多进程并行处理这些任务。我们创建了一个进程池，通过 `pool.imap_unordered` 方法并发执行 `process_word_positions` 函数，该函数对每个任务进行处理。`process_word_positions` 函数的作用是：对于给定的单词和文档文本，使用正则表达式查找该单词在文档中出现的所有位置（起止索引）。然后返回（单词，文档索引，值），其中值是一个元组（文档索引，出现次数，位置列表）。

在主进程中，我们收集并行处理的结果，并更新到 `result` 字典中。结构为：`result[word][doc_idx] = (doc_idx, count, positions)`。这样，函数遍历完所有的任务后，`result` 对象就包含了完整的倒排索引信息，其中每个词对应一个字典，该字典的键是文档ID，值是一个三元组（文档ID，该词在文档中出现的次数，以及所有出现的位置列表）。

最后，函数将生成的倒排索引保存到缓存文件中，以便后续使用。返回的倒排索引 `result` 是一个嵌套字典：外层字典的键是单词，内层字典的键是文档ID，值是一个三元组（文档ID，出现次数，位置列表）。

2.3.2 search.py

2.3.2.1 搜索匹配算法

本文件的作用是对用户输入的自然语言进行搜索匹配，并通过多种评估方法来进行相关度排序，下面是对搜索匹配算法和评估算法的详细说明：

```

1  # 处理搜索词，转为小写
2  search_str = search_str.lower()
3  s_list = search_str.split()
4

```

```

5 # 对每个搜索词在倒排索引中查找 - 使用倒排索引进行快速预筛选
6 temp = []
7 for s in s_list:
8     if s in inverse_index:
9         temp.append(inverse_index[s])
10    else:
11        temp.append({})
12 # 收集匹配的文档ID
13 candidate_docs = set()
14 for t in temp:
15     candidate_docs.update(t.keys())
16
17 # 如果没有找到任何匹配文档，尝试使用相似度搜索
18 if not candidate_docs and len(s_list) > 0:
19     try:
20         search_vec = bag.transform([search_str]).toarray()
21         similarity_scores = []
22         for i in range(len(text_list)):
23             sim = get_similarity(search_vec[0], count[i].A[0])
24             if sim > 0.1: # 只保留相似度超过阈值的文档
25                 similarity_scores.append((i, sim))
26             similarity_scores.sort(key=lambda x: -x[1])
27             candidate_docs = {idx for idx, _ in similarity_scores[:10]}
28     except Exception as e:
29         print(f"相似度搜索失败: {e}")

```

首先利用倒排索引进行预筛选，快速识别出包含任意查询词的文档，时间复杂度接近 $O(1)$ ，对于无精确匹配的情况，回退到向量空间模型进行相似度计算：使用 `bag.transform()` 将查询转换为向量空间表示，通过 `get_similarity()` 计算每个文档向量与查询向量的余弦相似度，设置0.1的相似度阈值排除无关文档，避免计算资源浪费，保留相似度最高的前10个文档作为候选集，平衡召回率与计算效率，此阶段融合倒排索引的布尔模型和向量模型的相似度算法，形成互补机制，确保不同场景下都有候选文档输出。

2.3.2.2 BM25相关评分

```

1 def compute_bm25_scores(query_tokens, doc_tokens_list, k1=1.5, b=0.75):
2     N = len(doc_tokens_list)
3     doc_lengths = [len(doc) for doc in doc_tokens_list]
4     avg_doc_len = sum(doc_lengths) / N if N > 0 else 0
5
6     query_term_counts = Counter(query_tokens)
7     query_terms = list(query_term_counts.keys())
8
9     df = {}
10    for term in query_terms:
11        df[term] = sum(1 for doc in doc_tokens_list if term in doc)
12
13    idf = {}
14    for term in query_terms:
15        idf[term] = math.log((N - df[term] + 0.5) / (df[term] + 0.5) + 1.0)
16
17    scores = []
18    for i, doc in enumerate(doc_tokens_list):
19        score = 0.0
20        doc_term_counts = Counter(doc)

```

```

21     doc_len = doc_lengths[i]
22     for term in query_terms:
23         if term in doc_term_counts:
24             tf = doc_term_counts[term]
25             term_score = idf.get(term, 0) * ((tf * (k1 + 1)) /
26                                             (tf + k1 * (1 - b + b *
doc_len / avg_doc_len)))
27             score += term_score
28             scores.append(score)
29     return scores

```

BM25作为搜索匹配的核心算法，其实现基于概率模型：

- **逆文档频率(IDF)**计算：采用平滑处理公式 $\text{math.log}((N - \text{df} + 0.5) / (\text{df} + 0.5) + 1)$ 避免除零错误
- **词频(TF)**归一化：使用 $k1=1.5$, $b=0.75$ 的标准参数配置，控制词频饱和度和文档长度影响
- **文档长度补偿**：通过 $b * \text{doc_len} / \text{avg_doc_len}$ 因子对长文档进行惩罚，抑制文档长度带来的统计偏差
- 参数设计：
 - $k1$ 控制词频的饱和度（默认1.5）
 - b 控制文档长度归一化强度（默认0.75）
 - 对数公式避免罕见词获得过高的IDF值

此算法精确量化查询词与文档的统计相关性，为后续结果排序提供客观依据。

2.3.2.3 短语匹配增强

```

1  # 检查短语匹配 - 使用安全的方式
2  if len(s_list) > 1:
3      try:
4          for file_index, item in list(result_dict.items()):
5              if item.count >= len(s_list):
6                  full_text = text_list[file_index].lower()
7                  if ' '.join(s_list) in full_text:
8                      item.is_phrase_match = True
9                      item.bm25_score *= 2.0
10                 try:
11                     for m in re.finditer(r'\b{}\b'.format(re.escape('
'.join(s_list)))), full_text):
12                         item.occurrence.insert(0, m.span())
13                 except:
14                     pass
15             except Exception as e:
16                 print(f"处理短语匹配时出错: {e}")

```

针对前面预筛选分割词组和句子进行匹配导致最终匹配并不完全的问题：

- 采用边界检测 $\backslash b\{\}\backslash b$ 确保精确匹配完整短语而非词袋组合
- 通过 `re.escape()` 对查询词进行安全转义，避免特殊字符导致的正则错误
- 双阶段检测逻辑：
 1. 快速字符串包含检测 `in full_text` 筛选候选文档
 2. 精确边界正则匹配确定短语位置

- 对短语匹配结果应用2.0倍BM25分数加权，显著提升排序位置
- 将短语匹配位置置于位置列表前端，优化结果摘要生成

2.3.2.4 评估与排序算法

结果评估系统采用混合评分策略，综合多个维度的信号生成最终排序。

计算如下：

```
1 raw_score = (  
2     item.bm25_score * 25 +  
3     item.similarity * 30 +  
4     item.count * 15 +  
5     (30.0 if item.is_phrase_match else 0) +  
6     ocr_bonus  
7 )  
8  
9 # 计算匹配比例惩罚因子  
10 match_ratio = min(1.0, item.count / max(1, query_term_count))  
11 match_penalty = 0.3 + 0.7 * match_ratio  
12  
13 item.total_relevance = raw_score * match_penalty  
14 item.total_relevance = min(100, item.total_relevance * 100 / max_possible)
```

平衡了统计显著性、语义相关性和查询意图满足度，对部分匹配结果应用0.3-1.0动态折扣，避免单一模型的局限性，再进行排序：

```
1 result_list.sort(key=lambda x: (  
2     -x.count/max(1, len(s_list)),  
3     -x.total_relevance,  
4     -x.bm25_score  
5 ))
```

最终排序采用三级优先策略：

1. **查询覆盖度优先：** count/query_term_count 确保首选完全匹配文档
2. **综合相关性降序：** total_relevance 为主要排序依据
3. **BM25得分决胜：** 相同相关性时按统计分数二次排序

2.3.3 main.py

main.py 主要负责使用 Gradio 库构建可视化界面以及调用前两个文件的方法，并提供用户评估方法，因此在这里不作介绍，可直接查看源码。

2.4 创新以及算法优化

2.4.1 多媒体实现

利用OCR技术读取图片内容，将图片与文字信息绑定：


```

1  def get_text_list_with_ocr():
2      photo_metadata = process_photos()
3      photo_ocr_map = {item['photo_path']: item['ocr_text'] for item in
photo_metadata}
4
5      for file_path in file_paths:
6          related_photos = file_to_photos.get(file_path, [])
7          if related_photos:
8              ocr_texts = [photo_ocr_map.get(photo, "") for photo in
related_photos]
9              if ocr_texts:
10                 ocr_content = " ".join(ocr_texts)
11                 text_content += f"\n[OCR_TEXT] {ocr_content}"

```

使用 [OCR_TEXT] 标识符将图片OCR内容嵌入文本特征，实现"以图搜文"和"图文互检"的双向检索能力，OCR匹配结果在搜索结果中高亮显示，实现跨模态结果融合。

2.4.2 缓存加速

考虑到每次进入系统都需要初始化系统并进行词袋模型以及倒排索引的构建的话，会浪费太多时间，所以我设计系统能够把每次构建的信息全部存储下来，当文件数据有更新变动时删除并更新缓存，这样每次进入系统就可以直接加载缓存实现快速启动，对于数据的处理如倒排索引的构建也采用进程池并行加速计算。

2.4.3 搜索算法优化

为了防止搜索算法搜索信息不全面，我对搜索匹配算法进行了一定的优化，采用三层递进式搜索：先使用倒排索引快速筛选，再通过相似度进行搜索，最后通过BM25相关性评分查找来确保召回率。

```

1  def run_search(search_str, inverse_index, files, text_list, bag, count):
2      # 1. 倒排索引快速筛选
3      candidate_docs = set()
4      for t in temp:
5          candidate_docs.update(t.keys())
6
7      # 2. 相似度搜索备用方案
8      if not candidate_docs:
9          search_vec = bag.transform([search_str]).toarray()
10         # ...计算相似度...
11
12     # 3. BM25相关性评分
13     bm25_scores = compute_bm25_scores(query_tokens, doc_tokens_list)

```

2.4.4 评估算法优化

在测试过程中发现程序对于句子和短语的处理逻辑是这样的：拆分成单词然后去寻找所有单词的出现文档，这显然是将范围扩大了，所以我采用了边界检测 `\b{}\b` 确保精确匹配完整短语而非词袋组合；通过 `re.escape()` 对查询词进行安全转义，避免特殊字符导致的正则错误；对短语匹配结果应用2.0倍BM25分数加权，显著提升排序位置；将短语匹配位置置于位置列表前端，优化结果摘要生成；对不完全匹配的短语设置惩罚机制：

```

1 # 计算匹配比例惩罚因子 - 部分匹配将受到惩罚
2 match_penalty = 0.3 + 0.7 * match_ratio # 范围从0.3到1.0
3 # 应用匹配比例惩罚
4 item.total_relevance = raw_score * match_penalty

```

2.4.5 评估算法反馈

```

1     if feedback and (len(feedback) > 0 or (feedback_features and
feedback_terms)):
2         adjust_scores_with_feedback(result_list, feedback,
feedback_features, count, bag, text_list, feedback_terms)
3
4     result_list.sort(key=lambda x: (-x.count / max(1, len(s_list)), -
x.total_relevance, -x.bm25_score))
5
6     # 阈值过滤
7     base_threshold = 20.0
8     if len(s_list) > 1:
9         base_threshold += 10.0
10    not_relevant_count = sum(1 for v in feedback.values() if v == 0) if
feedback else 0
11    if feedback:
12        threshold_boost = min(50, not_relevant_count * 5) / 100
13        base_threshold *= (1 + threshold_boost)
14    filtered_results = [item for item in result_list if item.total_relevance
>= base_threshold]
15    if len(s_list) > 1 and len(filtered_results) > 5:
16        multi_word_matches = [item for item in filtered_results if
item.count > 1]
17        if len(multi_word_matches) >= 3:
18            filtered_results = multi_word_matches
19
20    # --- 动态最大返回数调整 ---
21    temp_max_results = query_settings[search_str]["max_results"]
22    print(f"当前最大返回数: {temp_max_results}, 搜索词: {search_str}")
23
24    if feedback:
25        relevant_count = sum(1 for v in feedback.values() if v == 1)
26        not_relevant_ratio = sum(1 for v in feedback.values() if v == 0) /
max(1, len(feedback))
27        if relevant_count > 0:
28            temp_max_results += int(relevant_count * 3)
29        if not_relevant_ratio > 0.2:
30            temp_max_results = max(3, temp_max_results -
int(not_relevant_count * 2))
31    # 更新设置
32    query_settings[search_str]["params"] = current_params.copy()
33    query_settings[search_str]["max_results"] = temp_max_results
34
35    return filtered_results[:temp_max_results]
36
37
38 def adjust_scores_with_feedback(result_list, feedback, feedback_features,
count, bag, text_list, feedback_terms=None):

```

```

39     """根据用户反馈调整搜索结果的分值"""
40     if not feedback or not result_list:
41         return
42
43     # 1. 调整已有评估的文档分值
44     for item in result_list:
45         doc_index = item.index
46
47         # 对已经评估过的文档进行分值调整
48         if doc_index in feedback:
49             if feedback[doc_index] == 1: # 标记为相关
50                 # 提高相关文档的分值
51                 item.total_relevance = min(100, item.total_relevance * 1.5)
52                 item.feedback_adjusted = True
53                 item.has_feedback = True
54             else: # 标记为不相关
55                 # 降低不相关文档的分值
56                 item.total_relevance = max(0, item.total_relevance * 0.5)
57                 item.feedback_adjusted = True
58                 item.has_feedback = True
59
60         # 2. 使用相关性反馈特征向量调整其他文档的分值
61         if feedback_features and 'positive' in feedback_features and
62         feedback_features.get('count_pos', 0) > 0:
63             pos_features = feedback_features['positive'] /
64             feedback_features['count_pos']
65
66             # 计算平均不相关文档特征向量
67             neg_features = None
68             if 'negative' in feedback_features and
69             feedback_features.get('count_neg', 0) > 0:
70                 neg_features = feedback_features['negative'] /
71                 feedback_features['count_neg']
72
73             # 对每个搜索结果计算与相关文档的相似度
74             for item in result_list:
75                 doc_index = item.index
76                 if doc_index not in feedback: # 只处理未评估的文档
77                     try:
78                         # 获取文档特征向量
79                         doc_vector = count[doc_index].A[0]
80
81                         # 计算与相关文档的相似度
82                         pos_similarity = cosine_similarity(doc_vector,
83                         pos_features)
84
85                         # 如果有不相关文档，计算与不相关文档的相似度并减去
86                         neg_similarity = 0
87                         if neg_features is not None:
88                             neg_similarity = cosine_similarity(doc_vector,
89                             neg_features)
90
91                         # 计算综合相似度 (Rocchio算法简化版)
92                         feedback_similarity = pos_similarity - 0.5 *
93                         neg_similarity

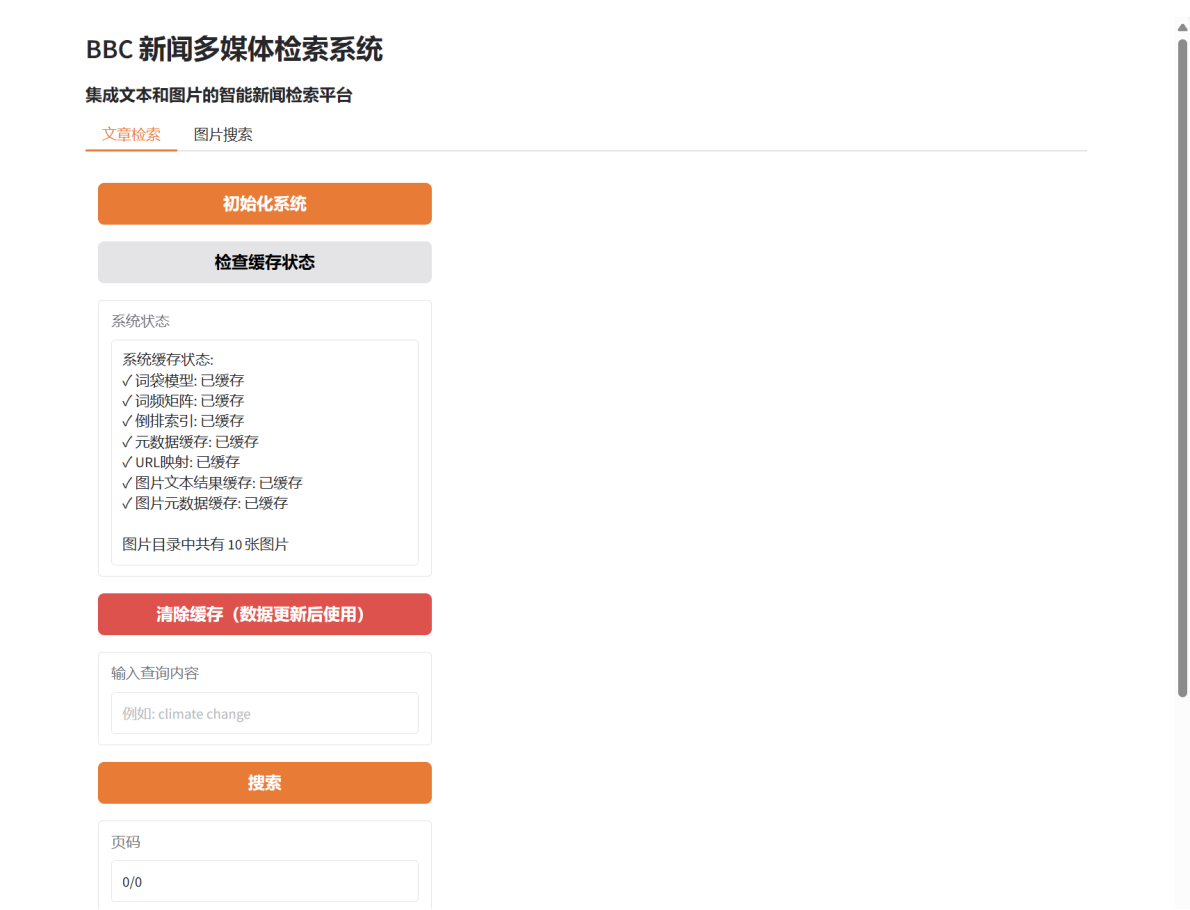
```

```
87
88         # 调整文档得分
89         feedback_boost = max(0, feedback_similarity * 25) # 将相
           似度转换为加分
90         item.total_relevance = min(100, item.total_relevance +
           feedback_boost)
91
92         if feedback_boost > 0:
93             item.feedback_adjusted = True
94     except Exception as e:
95         print(f"反馈调整得分时出错：{e}")
```

根据用户的评价结果，对查询的BM25参数以及返回的文档数目进行相应的调整和优化，并重新计算文档相似度。

3. 结果展示

进入系统初始页面：



可以选择检查缓存，重新生成或者直接载入现有缓存，

BBC 新闻多媒体检索系统

集成文本和图片的智能新闻检索平台

文章检索

图片搜索

初始化系统

检查缓存状态

系统状态

正在加载BBC新闻数据...
处理图片数据...
成功处理 10 张图片的文本
成功加载 162 篇文档 (用时: 0.10秒)
加载文章链接信息...
成功加载 162 个链接 (用时: 0.00秒)
从缓存加载词袋模型...
词袋模型包含 5292 个特征词 (用时: 0.01秒)
从缓存加载倒排索引...
索引加载完成! 共为 5292 个词条创建了索引 (用时 0.08秒)

系统就绪! 请在搜索框中输入查询内容

清除缓存 (数据更新后使用)

输入查询内容

例如: climate change

搜索

页码

0/0

接下来进行文本检索展示，首先检索单词 change：

BBC 新闻多媒体检索系统

集成文本和图片的智能新闻检索平台

文章检索

图片搜索

初始化系统

检查缓存状态

系统状态

正在加载BBC新闻数据...
处理图片数据...
成功处理 10 张图片的文本
成功加载 162 篇文档 (用时: 0.10秒)
加载文章链接信息...
成功加载 162 个链接 (用时: 0.00秒)
从缓存加载词袋模型...
词袋模型包含 5292 个特征词 (用时: 0.01秒)
从缓存加载倒排索引...
索引加载完成! 共为 5292 个词条创建了索引 (用时 0.08秒)

系统就绪! 请在搜索框中输入查询内容

清除缓存 (数据更新后使用)

输入查询内容

change

搜索

页码

1/16

← 上一篇

下一篇 →

✓ 评估为相关

X 评估为不相关

评估结果

找到 16 条相关结果 (搜索用时: 0.04秒)

Mexico sues Google over 'Gulf of America' name change.txt

类型: 文章

链接: <https://www.bbc.co.uk/news/articles/c5yk5nj7p7kq>

总体相关度: 66.4/100

详细指标: BM25评分: 0.06, 匹配词数: 1.0, 词频: 5.00

余弦相似度: 0.1104

内容片段:

...baum has asked the tech company multiple times to change the name Mexico is suing Google for ignoring rep...

... for the body of water to be renamed, arguing the change was justified because the US "do most of the work...

...egal action. At the time, Google said it made the change as part of "a longstanding practice" of following...

...gal action. At the time, Google said it made the change as part of "a longstanding practice" of following...

可以看到有相应的语法高亮以及相关度展示，同时按照相关度指标进行排序展示，同时也会展示相关的图片，我们可以对相应的检索结果进行评估：

BBC 新闻多媒体检索系统

集成文本和图片的智能新闻检索平台

文章检索 图片搜索

初始化系统

检查缓存状态

系统状态

正在加载BBC新闻数据...

处理图片数据...

成功处理 10 张图片的文本

成功加载 162 篇文章 (用时: 0.10秒)

加载文章链接信息...

成功加载 162 个链接 (用时: 0.00秒)

从缓存加载词袋模型...

词袋模型包含 5292 个特征词 (用时: 0.01秒)

从缓存加载倒排索引...

索引加载完成! 共为 5292 个词条创建了索引 (用时 0.08 秒)

系统就绪! 请在搜索框中输入查询内容

清除缓存 (数据更新后使用)

输入查询内容

month

搜索

页码

6/35

← 上一篇

下一篇 →

✓ 评估为相关

✗ 评估为不相关

评估结果

已将文档《Liberal Party names first female leader after historic Australia election loss.txt》评估为【相关】

当前评估统计: 相关 4, 不相关 0, 准确率 100.00%

Ancient Indian skeleton gets a museum home six years after excavation.txt

类型: 文章
链接: <https://www.bbc.co.uk/news/articles/c20nk27lmg3o>
总体相关度: 57.4/100
详细指标: BM25评分: 0.02, 匹配词数: 1.0, 词频: 2.00
余弦相似度: 0.0546

内容片段:

...t was excavated.The BBC had reported earlier this month that the skeleton had been left inside an unprote...
...was excavated. The BBC had reported earlier this month that the skeleton had been left inside an unprote...

相关图片:

Three alleged Iranian spies have appeared in court charged with targeting UK-based journalists so that "serious violence" could be inflicted on them. ...

Ancient Indian skeleton gets a museum home six years after excavation.png

文本:

以下是短语级检索展示，对检索结果进行语法高亮标注，然后进行短语级匹配以防错误：

初始化系统

检查缓存状态

系统状态

正在加载BBC新闻数据...
处理图片数据...
成功处理 10 张图片的文本
成功加载 162 篇文档 (用时: 0.10秒)
加载文章链接信息...
成功加载 162 个链接 (用时: 0.00秒)
从缓存加载词袋模型...
词袋模型包含 5292 个特征词 (用时: 0.01秒)
从缓存加载倒排索引...
索引加载完成! 共为 5292 个词条创建了索引 (用时 0.08 秒)

系统就绪! 请在搜索框中输入查询内容

清除缓存 (数据更新后使用)

输入查询内容

young person

搜索

页码

1/39

← 上一篇

下一篇 →

✓ 评估为相关

X 评估为不相关

评估结果

已将文档《Liberal Party names first female leader after historic Australia election loss.txt》评估为【相关】
当前评估统计: 相关 4, 不相关 0, 准确率 100.00%

找到 39 条相关结果 (搜索用时: 0.08秒)

'Proud to be young' - Beauty queen, lawyer and Botswana's youngest cabinet minister.txt

类型: 文章

链接: <https://www.bbc.co.uk/news/articles/c1mev5yqk0yo>

总体相关度: 100.0/100

详细指标: BM25评分: 3.93, 匹配词数: 2.0, 词频: 25.00

余弦相似度: 0.2102

✓ 短语精确匹配

内容片段:

...s headquarters in the capital, Gaborone. "I'm a **young person** living in Botswana, passionate about youth develo...

...ry's headquarters in the capital, Gaborone."I'm a **young person** living in Botswana, passionate about youth develo...

... the BBC ahead of his inauguration that he wanted **young** people to be the solution - "to become entreprene...

...uitable ambassador - and Chombo was clearly it: a **young** woman already committed to various causes. He made...

...outh and gender."I've never been more proud to be **young**," she told the BBC at the ministry's headquarters. ...

相关图片:

To the blast of a trumpet and the beating of drums, Fatima Hazzouri has come home. Thirteen years after civil war forced her to leave, she's back in her native city, Hama in Syria, blending in the revolution as she steps off a bus crammed with returning women and children, part of a long convo of civilians and trouble.

In a central square, they're greeted ecstatically by musicians and dancers in embroidered silk shirts.

'Proud to be young' - Beauty queen, lawyer and Botswana's youngest cabinet minister.png

文本:

To the blast of a trumpet and the beating of drums, Fatima Hazzouri has come home. Thirteen years after civil war forced her to leave, she's back in ...

以下是评估进行的参数调整:

集成文本和图片的智能新闻检索平台

初始化系统

检查缓存状态

系统状态

正在加载BBC新闻数据...
处理图片数据...
成功处理 10 张图片的文本
成功加载 162 篇文档 (用时: 0.04秒)
加载文章链接信息...
成功加载 162 个链接 (用时: 0.00秒)
从缓存加载词袋模型...
词袋模型包含 5292 个特征词 (用时: 0.01秒)
从缓存加载倒排索引...
索引加载完成! 共为 5292 个词条创建了索引 (用时 0.16 秒)

系统就绪! 请在搜索框中输入查询内容

清除缓存 (数据更新后使用)

输入查询内容

climate change

搜索

页码

6/17

← 上一篇

下一篇 →

✓ 评估为相关

X 评估为不相关

已将文档《Military rulers in Mali dissolve all political parties.txt》评估为【不相关】

当前评估统计: 相关 2, 不相关 4, 准确率 33.33%

- ✓ 检索参数已根据您的反馈自动更新
- o k1: 1.55 (控制词频重要性)
 - o b: 0.69 (文档长度归一化)
 - o 权重分配: 相似度(30.0), BM25(24.9), 匹配词(15.1)

Military rulers in Mali dissolve all political parties.txt

类型: 文章

链接: <https://www.bbc.co.uk/news/articles/cqj7z92xw9nq>

总体相关度: 42.5/100

详细指标: BM25评分: 0.15, 匹配词数: 1.0, 词频: 2.00

余弦相似度: 0.0277

内容片段:

...ry-run states leave West African bloc - what will **change**?Why young Africans are celebrating military takeo...

...ry-run states leave West African bloc - what will **change**? Why young Africans are celebrating military tak...

以下是图片检索，同时也会展示相关的文章和链接：

文章检索 图片搜索

搜索图片文本

搜索图片

month

搜索图片

图片搜索说明

系统会识别图片中的文字内容，您可以：

- 搜索图片中出现的特定文字或短语
- 查看识别出的完整文本
- 查看图片所关联的新闻文章

文本质量取决于图片清晰度和文字特征。

找到 3 张文本包含 'month' 的图片 (搜索用时: 0.00秒)

"Melania" appeared on the banks of the River Savin in July 2020, four months before her human inspiration left the White House. Now, four months after the erstwhile Melania Knays resumed residence at Washington's most famous address, her larger-than-life-size avatar has apparently made an undignified exit from her Slovenian hometown, Sevnica. All that remains of the massive bronze statue are the feet — and the two metro-tall

• The Melania statue in Sevnica, Slovenia, was unveiled in July 2020, four months before her human inspiration left the White House. Now, four months after the erstwhile Melania Knays resumed residence at Washington's most famous address, her larger-than-life-size avatar has apparently made an undignified exit from her Slovenian hometown, Sevnica. All that remains of the massive bronze statue are the feet — and the two metro-tall

Is Britain really inching back towards the EU.png

相关度得分: 60.0

OCR文本: "Melania" appeared on the banks of the River Savin in July 2020, four months before her human inspiration left the White House. Now, four months after the erstwhile Melania Knays resumed residence at Washington's most famous address, her larger-than-life-size avatar has apparently made an undignified exit from her Slovenian hometown, Sevnica. All that remains of the massive bronze statue are the feet — and the two metro-tall

关联文章: Is Britain really inching back towards the EU?

[查看文章](#)

A 1,000-year-old human skeleton which was buried sitting cross-legged in India has been moved to a museum six years after it was excavated. The BBC had reported earlier this month that the skeleton had been left inside an unprotected tarpaulin shelter close to the excavation site in western Gujarat state since 2019 because of bureaucratic wrangling.

On Thursday, the skeleton was shifted to a local museum, just a few miles away from where it was unearthed. Authorities say that it will be placed on display for the public after administrative procedures are completed.

'I'm overjoyed to be back' Syrians face daunting rebuild after years of war.png

相关度得分: 55.0

OCR文本: A 1,000-year-old human skeleton which was buried sitting cross-legged in India has been moved to a museum six years after it was excavated. The BBC had reported earlier this month that the skeleton had been left inside an unprotected tarpaulin shelter close to the excavation site in western Gujarat state since 2019 because of bureaucratic wrangling. On Thursday, the skeleton was shifted to a local museum — just a few miles away

Lancaster House and the article

On a warm morning earlier this month, a group of Metropolitan Police diplomatic protection officers sat in an anteroom off the ornate entrance hall in London's Lancaster House, sipping tea and nibbling chocolate biscuits, while upstairs a core group of European politicians discussed the future of European cooperation. It was an apt setting: everywhere you look in Lancaster House, there is evidence of the British government's efforts to rebuild its relationship with the EU. The British government has been working hard to rebuild its relationship with the EU. The British government has been working hard to rebuild its relationship with the EU.



Contact BBC News online - help, feedback and complaints.png

相关度得分: 55.0

OCR文本: Listen to Damian read this article On a warm morning earlier this month, a group of Metropolitan Police diplomatic protection officers sat in an anteroom off the ornate entrance hall in London's Lancaster House, sipping tea and nibbling chocolate biscuits, while upstairs a core group of European politicians discussed the future of European cooperation. It was an apt setting: everywhere you look in Lancaster House, there is evidence of the British government's efforts to rebuild its relationship with the EU. The British government has been working hard to rebuild its relationship with the EU.

关联文章: Contact BBC News online - help, feedback and complaints

[查看文章](#)

4. 实验总结

在本次实验中，我实现了一个信息检索系统，基于BM25算法和词向量相似度匹配实现了信息检索功能，通过OCR技术实现了多媒体模块，根据人工评估结果对BM25的参数权重进行了调整来优化检索结果并最终使用 Gradio 库实现了一个简洁美观的可视化界面，对信息检索的原理和实现有了进一步的了解，收获匪浅。